Exp 8 Write the Python to Implement Travelling Salesman Problem.

Aim:

To write a python program to implement travelling salesman problem.

Algorithm:

Step 1: Initialize the best distance and best route to be infinity and an empty list, respectively.

Step 2: Generate every possible permutation of the cities.

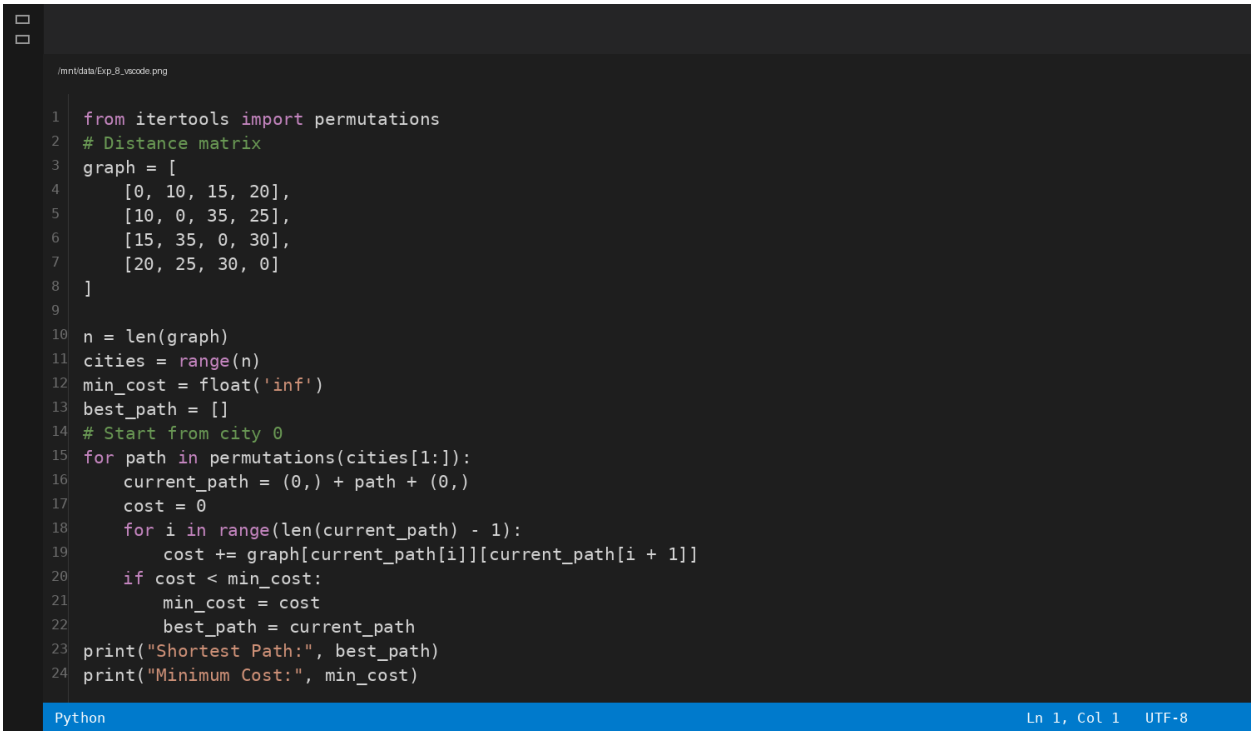
Step 3: For each permutation, calculate the distance of the path by summing the distances between each pair of consecutive cities in the permutation.

Step 4: Add the distance from the last city to the first city.

Step 5: If the calculated distance is shorter than the current best distance, update the best distance and best route.

Step 6: Return the best distance and best route.

Program:



from itertools import permutations

 # Distance matrix

graph = [

[0, 10, 15, 20],

[10, 0, 35, 25],

[15, 35, 0, 30],

[20, 25, 30, 0]

]

n = len(graph)

cities = range(n)

min_cost = float('inf')

best_path = []

```
# Start from city 0
for path in permutations(cities[1:]):
    current_path = (0,) + path + (0,)
    cost = 0

    for i in range(len(current_path) - 1):
        cost += graph[current_path[i]][current_path[i + 1]]

    if cost < min_cost:
        min_cost = cost
        best_path = current_path

print("Shortest Path:", best_path)
print("Minimum Cost:", min_cost)
```

Output:

Result:

Thus the Program was executed and the output was found to be Correct.

Exp 9

Write the python program to implement A* algorithm.

Aim:

To write a python program to implement A* algorithm.

Algorithm:

Step1 Initialize the open list with the starting node and the closed list as an empty set. Step 2: Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.

Step 3: While the open list is not empty, select the node with the lowest f score and set it as

the current node.

Step 4: If the current node is the goal node, return the reconstructed path.

Remove the current node from the open list and add it to the closed list.

Step 5: For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.

Step 6: If the neighbor is already in the closed list, skip it.

Step 7: If the neighbor is not in the open list, add it and set its g score and f score.

Step 8 :If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.

Step 9 : Otherwise, update the came from dictionary, g score, and f score of the neighbor.

Step 10 : Repeat steps 3-10 until the open list is empty.

If the goal node was never reached, return failure.

Program:

/mnt/data/Exp_9_vscode.png

```python
from queue import PriorityQueue
# Graph with cost
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 6)],
    'C': [('F', 5)],
    'D': [],
    'E': [('F', 2)],
    'F': []
}

# Heuristic values
heuristic = {
    'A': 7, 'B': 6, 'C': 2,
    'D': 1, 'E': 1, 'F': 0
}

def a_star(start, goal):
    pq = PriorityQueue()
    pq.put((0, start))
    cost = {start: 0}
    parent = {start: None}
    while not pq.empty():
        current = pq.get()[1]
        if current == goal:
            break
        for neighbor, weight in graph[current]:
            new_cost = cost[current] + weight
            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                priority = new_cost + heuristic[neighbor]
                pq.put((priority, neighbor))
                parent[neighbor] = current
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    print("Shortest Path:", path)
    print("Total Cost:", cost[goal])

# Driver code
a_star('A', 'F')
```

Python Ln 1, Col 1 UTF-8

from queue import PriorityQueue

 # Graph with cost

graph = {

'A': [('B', 1), ('C', 3)],

'B': [('D', 3), ('E', 6)],

'C': [('F', 5)],

'D': [],

```
    'E': [('F', 2)],
    'F': []
}
# Heuristic values
heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 1,
    'E': 1,
    'F': 0
}
def a_star(start, goal):
    pq = PriorityQueue()
    pq.put((0, start))
    cost = {start: 0}
    parent = {start: None}
    while not pq.empty():
        current = pq.get()[1]
        if current == goal:
            break
        for neighbor, weight in graph[current]:
            new_cost = cost[current] + weight
            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                priority = new_cost + heuristic[neighbor]
```

```
            pq.put((priority, neighbor))
            parent[neighbor] = current
    # Find path
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    print("Shortest Path:", path)
    print("Total Cost:", cost[goal])
# Driver code
a_star('A', 'F')
```

Output:

Result:

Thus, the Program was executed and the output was found to be Correct.

Exp 10

Write the python program for Map Coloring to implement CSP.

Aim:

To write a python program for map coloring to implement CSP.

Algorithm:

Initialize an empty assignment for the map coloring problem.

For each country in the map, add a variable X_i to the assignment.

Define the constraints as pairs of adjacent countries, and add a constraint (X_i, X_j) to the CSP such that X_i != X_j.

Define the domain of each variable to be the set of available colors.

Call the Backtrack function to find a consistent assignment for the variables.

In the Backtrack function, check if the assignment is complete. If it is, return the assignment.

Select an unassigned variable X_i from the assignment.

For each value v in the domain of X_i, check if it is consistent with the constraints and the assignment so far.

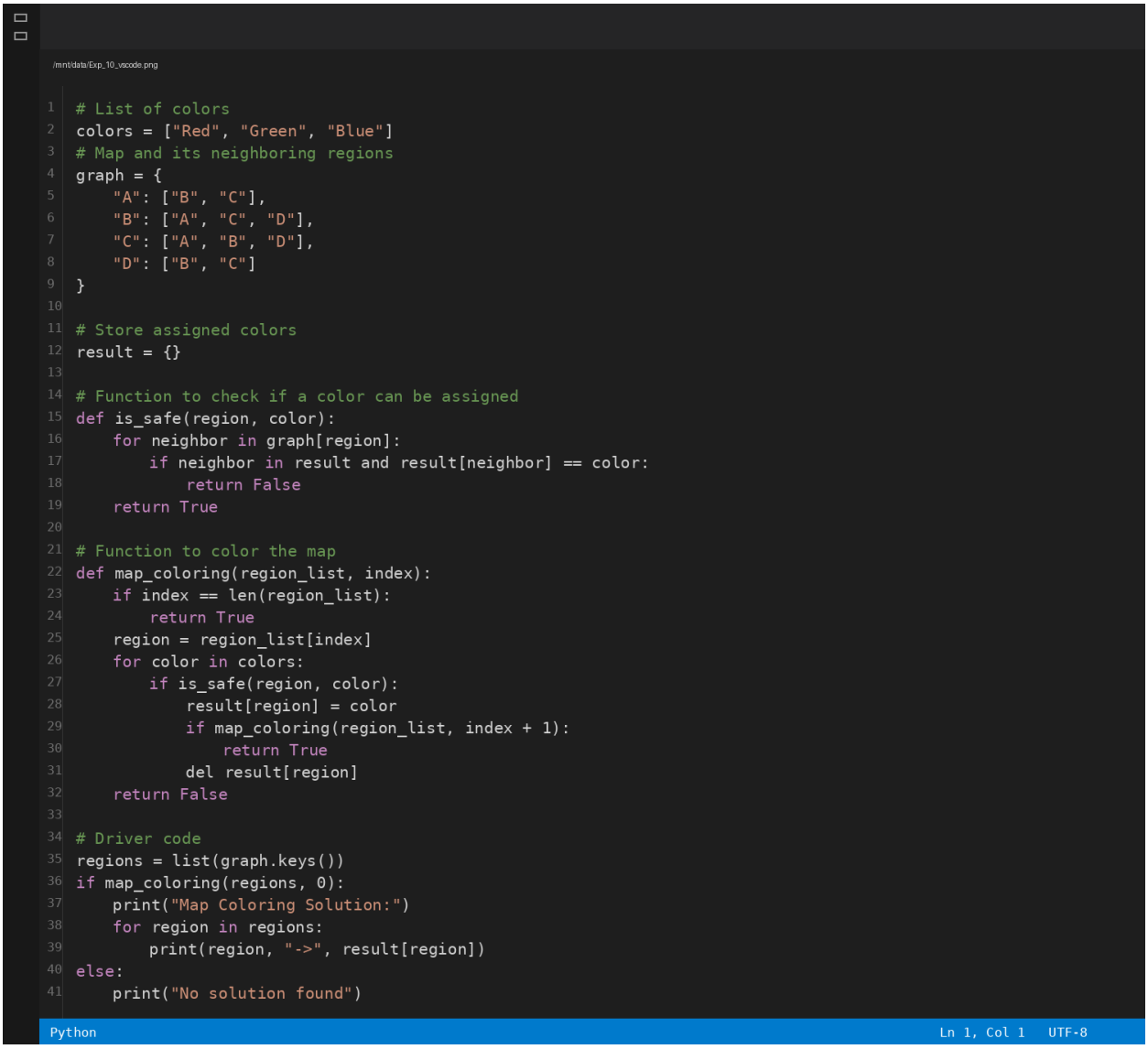
If it is, assign X_i = v to the assignment and recursively call Backtrack with the new assignment.

If the result of Backtrack is not a failure, return the result.

If the result is a failure, remove the assignment X_i = v and try the next value in the domain.

If all values in the domain have been tried and failed, return failure.

Program:



List of colors

colors = ["Red", "Green", "Blue"]

Map and its neighboring regions

graph = {

"A": ["B", "C"],

"B": ["A", "C", "D"],

"C": ["A", "B", "D"],

```
    "D": ["B", "C"]
}

# Store assigned colors
result = {}

# Function to check if a color can be assigned
def is_safe(region, color):
    for neighbor in graph[region]:
        if neighbor in result and result[neighbor] == color:
            return False
    return True

# Function to color the map
def map_coloring(region_list, index):
    if index == len(region_list):
        return True

    region = region_list[index]

    for color in colors:
        if is_safe(region, color):
            result[region] = color

            if map_coloring(region_list, index + 1):
                return True
```

```
            del result[region]

    return False

# Driver code
regions = list(graph.keys())

if map_coloring(regions, 0):
    print("Map Coloring Solution:")
    for region in regions:
        print(region, "->", result[region])
else:
    print("No solution found")
```

Output:

Result:

Hence, the simple python program for Map Coloring to implement CSP is done

Exp 11:

Write the python program for Tic Tac Toe game

Aim

To write and implement a Python program for the Tic Tac Toe game, where two players take turns placing X and O on a 3×3 board, and the program checks for a winner or a draw.

Algorithm

Start the program.

Create an empty 3×3 Tic Tac Toe board.

Assign Player 1 as X and Player 2 as O.

Display the board.

Ask the current player to enter the row and column position.

Check whether the selected position is empty.

If empty, place the player's symbol.

Otherwise, ask the player to enter another position.

Check if the current player has won by completing:

Any row,

Any column, or

Any diagonal.

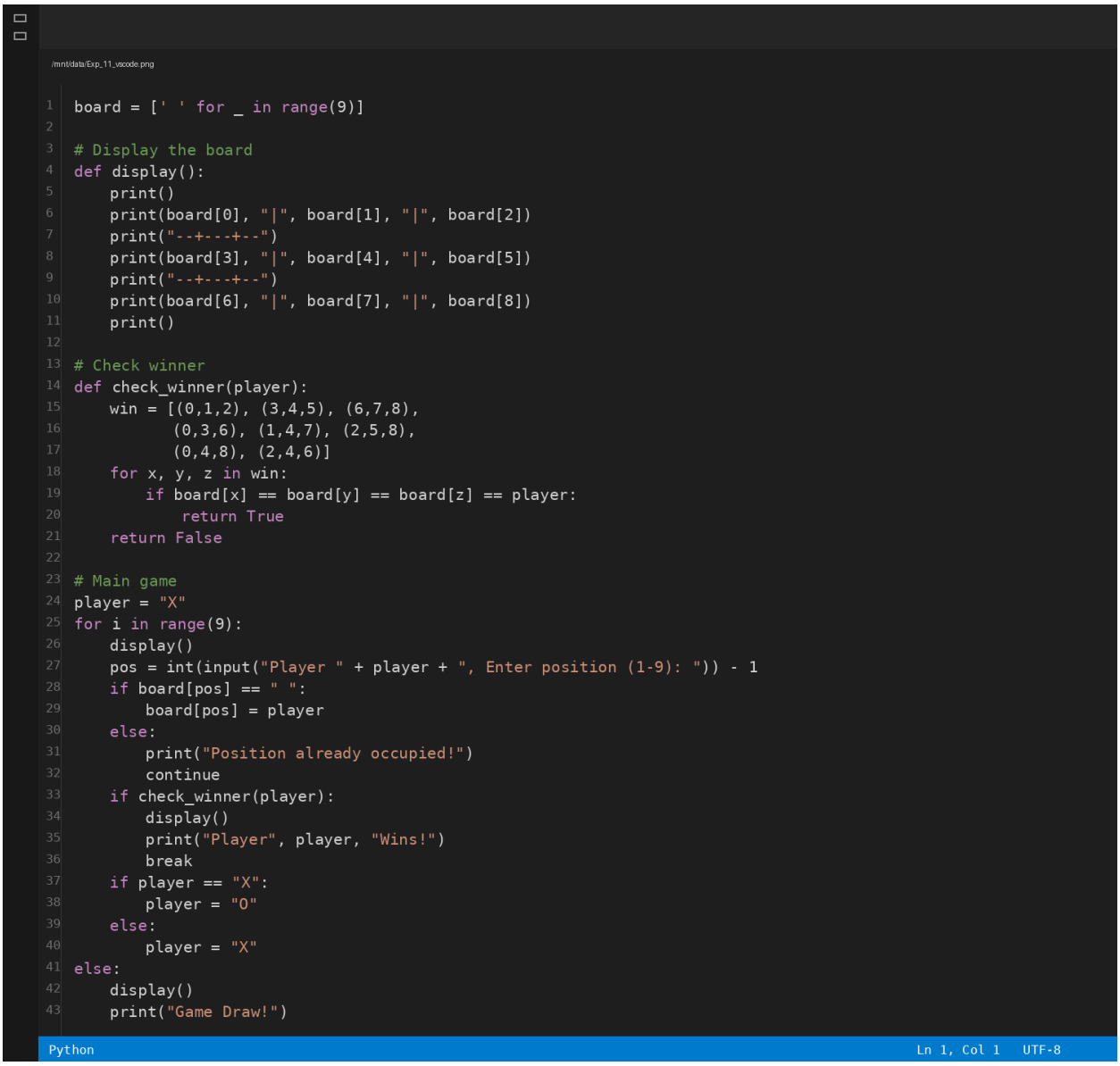
If a player wins, display the winner and stop the game.

If all 9 cells are filled and no player wins, declare the game as a Draw.

Otherwise, switch to the other player and repeat from Step 4.

End the program.

Program:



board = [' ' for _ in range(9)]

 # Display the board

def display():

print()

print(board[0], "|", board[1], "|", board[2])

print("--+---+--")

print(board[3], "|", board[4], "|", board[5])

```
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print()

# Check winner
def check_winner(player):
    win = [(0,1,2), (3,4,5), (6,7,8),
           (0,3,6), (1,4,7), (2,5,8),
           (0,4,8), (2,4,6)]

    for x, y, z in win:
        if board[x] == board[y] == board[z] == player:
            return True
    return False

# Main game
player = "X"

for i in range(9):
    display()
    pos = int(input("Player " + player + ", Enter position (1-9): ")) - 1

    if board[pos] == " ":
        board[pos] = player
    else:
        print("Position already occupied!")
```

```
        continue

    if check_winner(player):
        display()
        print("Player", player, "Wins!")
        break

    if player == "X":
        player = "O"
    else:
        player = "X"
else:
    display()
    print("Game Draw!")
```

Output:

Result:

The Python program for the Tic Tac Toe game was successfully implemented and executed. The program allowed two players to play alternately, correctly detected the winner or a draw, and displayed the final game result successfully.

Exp 12:

Write the python program to implement Minimax algorithm for gaming.

Aim

To write and implement a Python program for the Minimax algorithm, which helps a player choose the best possible move in a two-player game by maximizing its chances of winning while minimizing the opponent's chances.

Algorithm

Start the program.

Represent the game state (board or game tree).

Check whether the current state is a terminal state (win, lose, or draw).

If it is a terminal state, return the evaluation score.

If it is the maximizing player's turn:

Generate all possible moves.

Apply the Minimax function recursively for each move.

Choose the move with the highest score.

If it is the minimizing player's turn:

Generate all possible moves.

Apply the Minimax function recursively for each move.

Choose the move with the lowest score.

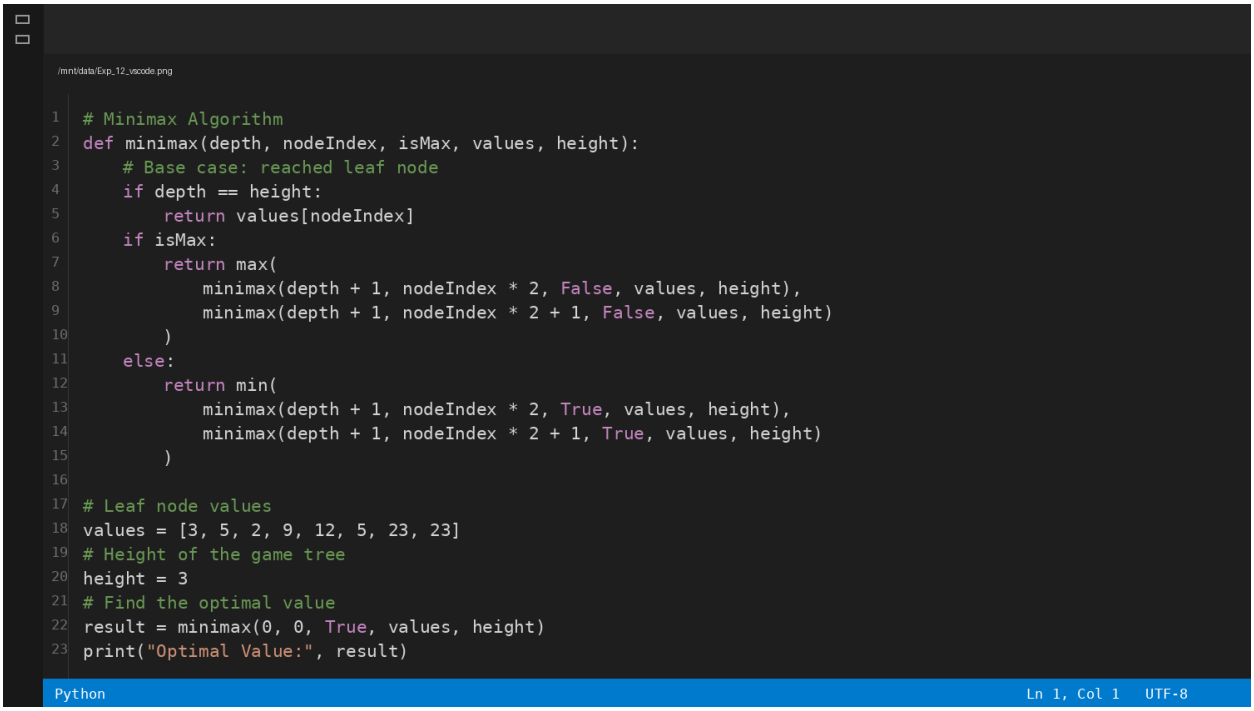
Repeat the process until all possible moves are evaluated.

Return the best move to the current player.

Display the optimal move and its score.

End the program.

Program:



 # Minimax Algorithm

def minimax(depth, nodeIndex, isMax, values, height):

 # Base case: reached leaf node

if depth == height:

return values[nodeIndex]

if isMax:

return max(

minimax(depth + 1, nodeIndex * 2, False, values, height),

minimax(depth + 1, nodeIndex * 2 + 1, False, values, height)

)

else:

return min(

```
        minimax(depth + 1, nodeIndex * 2, True, values, height),
        minimax(depth + 1, nodeIndex * 2 + 1, True, values, height)
    )

# Leaf node values
values = [3, 5, 2, 9, 12, 5, 23, 23]

# Height of the game tree
height = 3

# Find the optimal value
result = minimax(0, 0, True, values, height)

print("Optimal Value:", result)
```

Output:

Reault:

The Python program implementing the Minimax algorithm was successfully executed. The algorithm evaluated all possible game moves recursively and selected the optimal move, ensuring the best possible outcome for the current player.